

Navigating Codebases with `grep` and `find`

Introduction

As a software developer, you'll spend a significant amount of time reading and understanding code that already exists. Whether you're debugging a production issue, understanding how a feature works, or integrating new code into a large system, you need to **quickly find what you're looking for in the codebase**.

This lecture teaches you two essential Unix command-line tools: - **`grep`** - Search for text patterns within files - **`find`** - Locate files based on various criteria

Together, these tools will make you dramatically faster at navigating any codebase.

Why These Skills Matter

Imagine you're a new engineer at a company with a 500,000-line codebase. Your manager says: "There's a bug in the user authentication flow. Can you find where we validate passwords?"

Without `grep` and `find`, you'd be lost. With them, you can answer in seconds.

Real-world scenarios: - Finding all TODO comments in your code - Locating where a specific variable is defined - Understanding what files import a particular module - Identifying all database queries in your application - Finding all error messages related to a bug - Searching for deprecated function calls to replace

Part 1: The `find` Command

The `find` command locates files on your system based on criteria like name, type, size, or modification date.

Basic Syntax

```
find [path] [options] [expression]
```

Finding Files by Name

The most common use case is finding files by name pattern.

Example 1: Find all Java files

```
find . -name "*.java"
```

Breaking it down: - find - the command - . - start searching from current directory (. means “here”) - -name "*.java" - match files with names ending in .java

Output:

```
./sample_files/Main.java
./sample_files/Calculator.java
./sample_files/StringUtils.java
```

Example 2: Find all Python files

```
find . -name "*.py"
```

Output:

```
./sample_files/data_processor.py
./sample_files/text_utils.py
```

Example 3: Find files with a specific name

```
find . -name "config.txt"
```

Output:

```
./sample_files/config.txt
```

Finding Files by Type

Use -type to specify what kind of files you want.

Find all regular files (not directories)

```
find . -type f
```

Type options: - f - regular file - d - directory - l - symbolic link

Find only directories

```
find . -type d
```

Finding Files by Size

```
find . -size +10k      # Files larger than 10 kilobytes
find . -size -1M       # Files smaller than 1 megabyte
find . -size 100c      # Files exactly 100 bytes
```

Finding Files by Modification Time

```
find . -mtime 0        # Modified in the last 24 hours (today)
find . -mtime -7       # Modified within the last 7 days
find . -mtime +30      # Modified more than 30 days ago
```

Combining Conditions

You can combine multiple conditions with logical operators.

Find all Java files modified in the last week

```
find . -name "*.java" -mtime -7
```

Find all Python files larger than 1KB

```
find . -name "*.py" -size +1k
```

Find all files that are either Java or Python

```
find . \( -name "*.java" -o -name "*.py" \)
```

Note: The parentheses need to be escaped with backslashes: \ (and \)

Finding ALL file types in a codebase

When you want to locate all source code files:

```
find . \( -name "*.java" -o -name "*.py" -o -name "*.go" -o -name "*.ml" -o -name "*.lua" -o
```

Practical Exercise 1: Using find

Try these commands on the sample_files directory:

```
# Challenge 1: Find all Go files
```

```
find sample_files -name "*.go"
```

```
# Challenge 2: Find all OCaml files
```

```
find sample_files -name "*.ml"
```

```
# Challenge 3: Find all shell scripts
```

```
find sample_files -name "*.sh"
```

```
# Challenge 4: Find all configuration files
```

```
find sample_files -name "*.txt"
```

```
# Challenge 5: Find all Lua and Vimscript files
```

```
find sample_files \( -name "*.lua" -o -name "*.vim" \)
```

Part 2: The grep Command

The **grep** command searches for text **within files**. It scans files line by line and prints lines that match a pattern.

Basic Syntax

```
grep [options] [pattern] [files]
```

Finding Literal Text

The simplest use case is searching for a specific word or phrase.

Example 1: Find all TODO comments

```
grep "TODO" sample_files/*
```

Breaking it down: - `grep` - the command - `"TODO"` - the text to find (in quotes to be safe) - `sample_files/*` - search all files in `sample_files` directory

Output:

```
sample_files/Main.java:    * TODO: Add error handling for invalid input
sample_files/Main.java:    // TODO: Parse command line arguments
sample_files/Calculator.java:        // TODO: Add division by zero check
sample_files/data_processor.py:# TODO: Add logging functionality
```

The format is: `filename:matching_line`

Example 2: Find all import statements

```
grep "import" sample_files/*
```

This finds all lines containing the word “import” in any file.

Example 3: Find specific function calls

```
grep "System.out.println" sample_files/*.java
```

This finds all Java print statements.

Useful grep Options

-c : Count matches instead of showing them

```
grep -c "TODO" sample_files/*
```

Shows how many TODO comments in each file:

```
sample_files/Main.java:2
sample_files/Calculator.java:1
sample_files/data_processor.py:1
```

-n : Show line numbers

```
grep -n "TODO" sample_files/*
```

Output:

```
sample_files/Main.java:7:      * TODO: Add error handling for invalid input
sample_files/Main.java:14:     // TODO: Parse command line arguments
```

Now you know it's on lines 7 and 14 of Main.java.

-i : Case-insensitive search

```
grep -i "todo" sample_files/*
```

Finds "TODO", "Todo", "todo", etc.

-v : Invert match (find lines that DON'T match)

```
grep -v "TODO" sample_files/*.java
```

Finds all lines in Java files that don't contain "TODO".

-l : Show only filenames, not the matching lines

```
grep -l "version" sample_files/*
```

Output:

```
sample_files/Main.java
sample_files/data_processor.py
sample_files/list_utils.ml
```

This is useful when you just need to know which files contain something.

-r or -R : Search recursively through directories

```
grep -r "VERSION" sample_files/
```

Searches all files in sample_files and all subdirectories.

-C n : Show context (n lines before and after the match)

```
grep -C 2 "TODO" sample_files/Main.java
```

Shows 2 lines before and after each match, helping you understand the context.

-E : Extended regex (we'll cover this next!)

```
grep -E "pattern" sample_files/*
```

Enables more powerful pattern matching.

Practical Exercise 2: Using grep

Try these commands:

```
# Challenge 1: Find all FIXME comments
grep "FIXME" sample_files/*

# Challenge 2: Find all version-related lines
grep -i "version" sample_files/*

# Challenge 3: Count the number of matches
grep -c "import" sample_files/*.py

# Challenge 4: Find import statements and show line numbers
grep -n "import" sample_files/*.py

# Challenge 5: Find all files containing "database"
grep -l "database\|Database" sample_files/*
```

Part 3: Regular Expressions

Regular expressions (regex) allow you to search for patterns instead of literal text. This is where grep becomes truly powerful.

What is a Regular Expression?

A regex is a pattern that describes a set of strings. For example: - `[0-9]` matches any single digit - `[a-z]` matches any lowercase letter - `[a-zA-Z0-9_]` matches any letter, digit, or underscore

Character Classes

Regex uses special characters to match categories of text:

Pattern	Matches
.	Any single character
[abc]	Any single character: a, b, or c
[a-z]	Any lowercase letter
[A-Z]	Any uppercase letter
[0-9]	Any digit
[a-zA-Z0-9_]	Letter, digit, or underscore
\w	Word character (letter, digit, underscore)
\d	Digit
\s	Whitespace (space, tab, newline)
[^abc]	Any character EXCEPT a, b, or c

anchors

anchors match positions in the line, not characters:

Pattern	Matches
<code>^</code>	Start of line
<code>\$</code>	End of line
<code>^pattern\$</code>	Line containing only pattern

Example: Find lines that START with “public”

```
grep "^public" sample_files/*.java
```

Without the `^`, it would match “notpublic” or any line with “public” anywhere.

Example: Find lines that END with semicolon

```
grep ";$" sample_files/*.java
```

Quantifiers

Quantifiers specify how many times to match something:

Pattern	Matches
<code>a*</code>	Zero or more ‘a’ characters
<code>a+</code>	One or more ‘a’ characters
<code>a?</code>	Zero or one ‘a’ character
<code>a{3}</code>	Exactly three ‘a’ characters
<code>a{3,5}</code>	Between 3 and 5 ‘a’ characters
<code>a{3,}</code>	Three or more ‘a’ characters

Example: Find version numbers (like 1.2.3)

```
grep -E "[0-9]+\.[0-9]+\.[0-9]+" sample_files/*
```

Breaking it down: - `[0-9]+` - one or more digits - `\.` - literal dot (escaped because `.` is special) - `[0-9]+` - one or more digits - `\.` - literal dot - `[0-9]+` - one or more digits

Alternation

Use `|` (pipe) to match one pattern OR another. **Important: You must use `-E` flag!**

Example: Find import or export statements

```
grep -E "import|export" sample_files/*
```

Example: Find Java or Python comments

```
grep -E "//|#" sample_files/*
```

Putting It Together: Complex Patterns

Find variable assignments

```
grep -E "[a-zA-Z_][a-zA-Z0-9_]* *" sample_files/*
```

This matches: - `[a-zA-Z_]` - variable must start with letter or underscore - `[a-zA-Z0-9_]*` - followed by zero or more letters, digits, or underscores - `*` - optional spaces - `=` - equals sign

So it matches: `x=5`, `myVar = 10`, `_private_var=value`

Find email addresses

```
grep -E "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}" sample_files/*
```

This is complex but matches most email addresses!

Find function definitions in Python

```
grep -E "^def [a-z_]+\(" sample_files/*.py
```

This matches lines that: - Start with `def` (`^def`) - Have a space and function name in lowercase with underscores - End with opening parenthesis

Practical Exercise 3: Using Regular Expressions

Try these commands:

```
# Challenge 1: Find all lines with numbers  
grep "[0-9]" sample_files/*
```

```
# Challenge 2: Find lines starting with "public"  
grep "^public" sample_files/*.java
```

```
# Challenge 3: Find private methods (with optional whitespace)  
grep "^s*private" sample_files/*.java
```

```
# Challenge 4: Find camelCase words (lowercase then uppercase)  
grep -E "[a-z][A-Z]" sample_files/*
```

```
# Challenge 5: Find all function definitions (Java or Python)  
grep -E "(public|private|def )" sample_files/*
```

```
# Challenge 6: Find hex color codes  
grep -E "#[0-9A-Fa-f]{6}" sample_files/*
```



```
# Challenge 7: Find URLs
grep -E "https?://[^\ ]+" sample_files/*
```

Part 4: Combining find and grep

The real power comes from combining `find` and `grep`. This is how you navigate large codebases efficiently.

Pattern 1: Search only in specific file types

Find all TODOs in Java files only

```
find . -name "*.java" -exec grep -l "TODO" {} \;
```

This: 1. Uses `find` to locate all `.java` files 2. Uses `-exec` to run `grep` on each file found 3. The `{}` is replaced with each filename 4. The `;` ends the `exec` command

Find all imports in Python files

```
find . -name "*.py" | xargs grep "^import"
```

Alternative syntax using pipes and `xargs`.

Pattern 2: Search across multiple file types

Find all version strings in all source files

```
find . \( -name "*.java" -o -name "*.py" -o -name "*.go" \) | xargs grep -E "[0-9]+\.[0-9]+"
```

Pattern 3: Find in recently modified files

Find all TODOs in files modified today

```
find . -name "*.java" -mtime 0 | xargs grep "TODO"
```

Pattern 4: Count occurrences across a codebase

How many times is a specific function called?

```
grep -r "validateEmail" sample_files/ | wc -l
```

This: 1. `grep -r` recursively searches all files 2. Finds all lines with “validateEmail” 3. `wc -l` counts the number of lines

How many TODO comments in the entire codebase?

```
grep -r "TODO" sample_files/ | wc -l
```

Practical Exercise 4: Combining find and grep

Try these commands:

```
# Challenge 1: Find all function definitions in Python files
find sample_files -name "*.py" -exec grep "^def " {} +

# Challenge 2: Find version strings in configuration files
find sample_files -name "*.txt" | xargs grep -E "[0-9]+\.[0-9]+\.[0-9]+"

# Challenge 3: Count FIXME comments across all files
grep -r "FIXME" sample_files/ | wc -l

# Challenge 4: Find all class definitions in Java files
find sample_files -name "*.java" | xargs grep "^public class"

# Challenge 5: Find all functions in Go files
find sample_files -name "*.go" | xargs grep "^func "
```

Part 5: Real-World Scenarios

Now let's look at how you'd use these tools in actual development work.

Scenario 1: You find a bug related to error handling

The Problem: Users report crashes when they enter invalid input. You need to find all error handling code.

The Solution:

```
# Find all places where exceptions are thrown or caught
grep -r "try\|catch\|throw\|raise" sample_files/

# Or look specifically in Java
grep -r "catch\|throw" sample_files/*.java

# Find error messages
grep -r "Error\|Exception" sample_files/*.java
```

Scenario 2: You're refactoring a function called validateEmail

The Problem: You renamed the function to isValidEmail. Now you need to update all call sites.

The Solution:

```
# Find all references to the old name
grep -r "validateEmail" sample_files/
```

```
# Count how many places need updating
grep -r "validateEmail" sample_files/ | wc -l

# See the context around each call
grep -r -n "validateEmail" sample_files/
```

Scenario 3: You need to understand how the API works

The Problem: You're adding a new endpoint to the API server. You need to see how other endpoints are structured.

The Solution:

```
# Find all handler functions
grep -r "Handler" sample_files/*.go

# Find all routes
grep -r "Router\|Route" sample_files/*.go

# Look at the HTTP methods used
grep -r "GET\|POST\|PUT\|DELETE" sample_files/*.go
```

Scenario 4: You're migrating to a new version of a library

The Problem: A library changed its API. You need to find all uses of the old API.

The Solution:

```
# Find all imports of that library
grep -r "import json" sample_files/

# Find all function calls from that library
grep -r "json\." sample_files/

# Get the context (line numbers)
grep -rn "json\." sample_files/
```

Scenario 5: Code review - finding violations of coding standards

The Problem: Your team requires certain patterns. You need to check for violations.

The Solution:

```
# Check if anyone is using deprecated print functions
grep -r "System.out.println" sample_files/*.java
```

```
# Find TODO comments that might indicate unfinished work
grep -r "TODO\|FIXME\|HACK" sample_files/

# Find comments that might indicate confusion
grep -r "XXX\|???\|CONFUSED" sample_files/
```

Part 6: Advanced Tips and Tricks

Using pipes to chain commands

Pipes | allow you to feed the output of one command to another.

Find all Python files with TODOs, sorted by line count

```
grep -r "TODO" sample_files/*.py | cut -d: -f1 | sort | uniq -c
```

Ignoring certain files

Search everywhere except a specific directory

```
grep -r "TODO" sample_files --exclude-dir=.git
```

Ignore certain file types

```
grep -r "TODO" sample_files --exclude="*.pyc"
```

Using multiple patterns

Find lines matching EITHER pattern A OR pattern B

```
grep -E "error|warning|fatal" sample_files/*
```

Find lines matching pattern A but NOT pattern B

```
grep -E "import" sample_files/* | grep -v "^#"
```

This finds all imports except those starting with # (comments).

Save results to a file

Output all TODOs to a file for later review

```
grep -r "TODO" sample_files/ > todos.txt
```

Create a report with line numbers and filename

```
grep -rn "FIXME" sample_files/ | sort > fixme_report.txt
```

Part 7: Common Mistakes and How to Fix Them

Mistake 1: Forgetting to escape special characters in regex

Wrong:

```
grep "email@domain.com" sample_files/*
```

The `.` is special in regex (matches any character), so this would also match “emailXdomainYcom”.

Right:

```
grep "email@domain\.com" sample_files/*
```

Or use literal text (without regex):

```
grep -F "email@domain.com" sample_files/*
```

Mistake 2: Not using `-E` when using `|` (alternation)

Wrong:

```
grep "import|export" sample_files/*
```

This looks for the literal text “import|export”, not either word.

Right:

```
grep -E "import|export" sample_files/*
```

Mistake 3: Forgetting to quote patterns with spaces

Wrong:

```
grep public static sample_files/*.java
```

This searches for “public” and uses “static” as a filename!

Right:

```
grep "public static" sample_files/*.java
```

Mistake 4: Not restricting to file types, getting too many results

Wrong:

```
grep "version" sample_files/*
```

Searches everything and finds version in comments, strings, variable names, etc.

Right:

```
grep "^VERSION\|^version" sample_files/*.java
```

Only searches for version variable declarations.

Mistake 5: Not knowing where to search

Wrong:

```
grep "function_name" .
```

This only searches files in the current directory, not subdirectories.

Right:

```
grep -r "function_name" .
```

The `-r` flag searches recursively.

Part 8: Quick Reference Guide

Most Useful grep Commands

<code>grep "text" file</code>	<i># Find literal text</i>
<code>grep -r "text" directory/</code>	<i># Recursive search</i>
<code>grep -E "pattern" file</code>	<i># Use regex</i>
<code>grep -i "text" file</code>	<i># Case-insensitive</i>
<code>grep -n "text" file</code>	<i># Show line numbers</i>
<code>grep -c "text" file</code>	<i># Count matches</i>
<code>grep -l "text" file</code>	<i># Show only filenames</i>
<code>grep -v "text" file</code>	<i># Invert (exclude matches)</i>
<code>grep "text" file1 file2 file3</code>	<i># Search multiple files</i>
<code>grep -E "a b" file</code>	<i># OR operator (needs -E)</i>
<code>grep "^text" file</code>	<i># Start of line</i>
<code>grep "text\$" file</code>	<i># End of line</i>

Most Useful find Commands

<code>find . -name "*.java"</code>	<i># Find by name</i>
<code>find . -type f</code>	<i># Find files (not dirs)</i>
<code>find . -type d</code>	<i># Find directories</i>
<code>find . -name "*.java" -mtime -7</code>	<i># Modified in last 7 days</i>
<code>find . -name "*.java" -size +10k</code>	<i># Larger than 10KB</i>
<code>find . -name "*.java" xargs grep "TODO"</code>	<i># find + grep</i>

Most Useful Regex Patterns

<code>[0-9]</code>	<i># Any digit</i>
<code>[a-z]</code>	<i># Any lowercase letter</i>
<code>[A-Z]</code>	<i># Any uppercase letter</i>
<code>[a-zA-Z0-9_]</code>	<i># Letters, digits, underscore</i>
<code>.</code>	<i># Any character</i>
<code>^</code>	<i># Start of line</i>

```
$           # End of line
*           # Zero or more
+           # One or more
?           # Zero or one
{3}         # Exactly 3
{3,5}       # 3 to 5
a|b         # A or B
```

Practice Exercises

Now it's time to practice! Use the `CHALLENGES.md` file in the repository to complete 22 progressive challenges.

Getting Started

1. Clone the repository
2. Navigate to the directory
3. Open `CHALLENGES.md`
4. Start with Challenge 1 and work your way through

What You'll Learn

- **Challenges 1-6** - Basic literal text matching and file finding
- **Challenges 7-14** - Anchors, character classes, and quantifiers
- **Challenges 15-22** - Advanced patterns and complex queries
- **Bonus challenges** - Real-world combinations

Success Criteria

- You can find any text pattern in a codebase
 - You understand how to use anchors and character classes
 - You can combine find and grep for powerful searches
 - You know when to use -E for extended regex
 - You can solve problems by constructing appropriate grep/find commands
-

Summary

By mastering `grep` and `find`, you've gained the ability to:

1. **Quickly locate files** in a codebase
2. **Search for text patterns** across files
3. **Navigate unfamiliar code** efficiently
4. **Debug issues** by finding related code
5. **Understand how components interact** by finding all references

6. **Refactor safely** by finding all places that need updates

These are the skills that separate experienced developers from beginners. A developer who can quickly navigate and search a codebase is dramatically more productive.

Additional Resources

Online Regex Testers

- regex101.com - Interactive regex pattern tester
- regexr.com - Visual regex explorer with explanations

Official Documentation

- `man grep` - Manual page for grep (in terminal)
- `man find` - Manual page for find (in terminal)
- `man regex` - Regex pattern information

Common Patterns Database

Email (basic)

```
[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}
```

URL

```
https?://[^\s]+
```

Hex color

```
#[0-9A-Fa-f]{6}
```

Version number

```
[0-9]+\.[0-9]+\.[0-9]+
```

IP address (simplified)

```
[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}
```

Next Steps

1. **Complete the challenges** in the scavenger hunt
 2. **Try on real codebases** - practice with your own projects
 3. **Learn more regex** - once comfortable, explore advanced patterns
 4. **Learn sed and awk** - similar tools for more complex text processing
 5. **Master your IDE's search** - most IDEs use grep internally
-